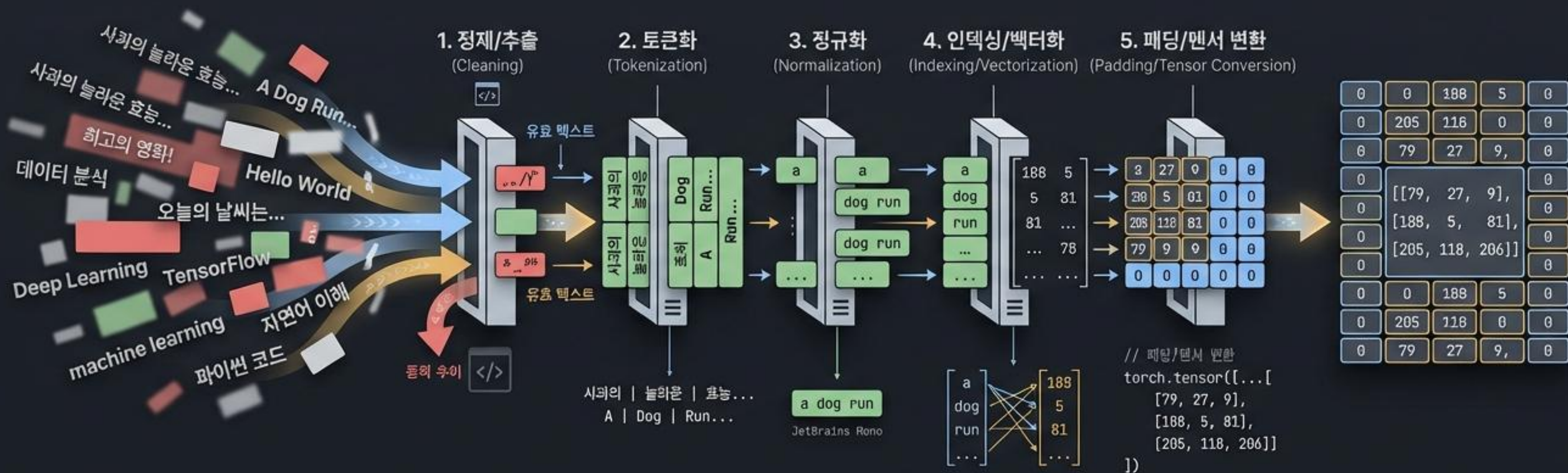


딥 러닝을 위한 자연어 처리 전처리 파이프라인 실습

텍스트 전처리 5단계 완벽 가이드 및 파이썬/파이토치 구현

원시 데이터부터 신경망 입력용 **텐서(Tensor)**까지의 변환 과정



텍스트 데이터의 신경망 입력을 위한 5단계 조립 라인

자연어 처리는 비정형 텍스트를 기계가 이해할 수 있는 정형화된 숫자로 변환하는 연속적인 과정입니다. 각 단계의 출력값(Output)은 다음 단계의 입력값(Input)으로 직접 연결됩니다.



[Step 1] 영어 토큰화: 공백 기준 분리의 용이성

- 영어는 띄어쓰기(Space) 단위로 단어 구분이 명확하여 토큰화가 비교적 수월합니다.
- spaCy, NLTK 등의 라이브러리 또는 파이썬의 기본 `.split()` 메서드로도 동일한 결과를 도출합니다.
- 더 작은 분석 단위가 필요할 경우 문자(Character) 단위 토큰화를 적용할 수 있습니다.

Python Code

```
en_text = 'A Dog Run back corner near spare bedrooms'  
spacy_en.tokenizer(en_text)  
word_tokenize(en_text)  
en_text.split()
```

Output (List)

```
['A', 'Dog', 'Run', 'back', 'corner', 'near',  
'spare', 'bedrooms']
```

문자(Character) 단위 토큰화 (`list(en_text)`)

```
['A', ' ', 'D', 'o', 'g', ' ', 'R', 'u', 'n', ' ', ...]
```

[Step 1] 한국어 띄어쓰기 토큰화의 치명적 한계

- 한국어는 교착어 특성상 어간에 조사나 접사가 붙어 형태가 지속적으로 변형됩니다.
- 단순 띄어쓰기로 분리할 경우, 동일한 의미의 단어가 다른 토큰으로 인식되어 단어 집합이 기하급수적으로 팽창합니다.

영어 (English) - 형태 유지

apple in a basket eat an apple apple is sweet



한국어 (Korean) - 조사의 결합

사과의 놀라운 효능... 사과를 먹으려고... 사과가 썩어서... 사과랑 오렌지...

`kor_text.split()`

['사과의', '놀라운', '...', '사과를',
'...', '사과가', '...', '사과랑', '...']

기계는 이를 4개의 완전히
다른 단어로 인식함

[Step 1] Mecab을 활용한 한국어 형태소 토큰화

- KoNLPy의 Mecab 형태소 분석기를 사용하여 어간과 조사를 완벽히 분리합니다.
- 의미를 가진 최소 단위(형태소)로 쪼개어, 본질적 의미를 하나의 단어로 정확히 매핑합니다.

Python Code

```
from konlpy.tag import Mecab

tokenizer = Mecab()
print(tokenizer.morphs(kor_text))
```

고유 단어 매핑 성공
(단일 토큰 인식)

Mecab Output (List)

['사과', '의', '놀라운', '효능', '이', '라는', '글', '을', '봤', '어', '.', '그래서', '오늘',
'사과', '를', '먹', '으려고', '했', '는데', '사과', '가', '씩', '어서', '슈퍼', '에', '가',
'서', '사과', '사과', '랑', '오렌지', '사', '왔', '어']

[Step 2] 단어 집합(Vocabulary) 생성: 데이터 정제 작업

- 단어 집합(Vocabulary): 중복을 제거한 텍스트 문치 내 총 고유 단어의 집합
- 실습 데이터: 네이버 영화 리뷰 분류 데이터 (ratings.txt)
- 정규 표현식(Regex) 정제: 분석에 불필요한 특수기호, 영문, 숫자 등을 제거하고 한글과 공백만 추출합니다.

Python Code

```
sample_data = data[:100]
sample_data['document'].str.replace(
    '[^ㄱ-ㅎㅏ-ㅣ가-힣 ]', '', regex=True)
```

index | document (Cleaned)

0		어릴 때 보고 지금 다시 봐도 재밌어요 ㅋㅋ
1		디자인을 배우는 학생 외국 디자이너 그 일군 전통을...
2		플리스 스토리 시리즈 부터 뉴 까지 버릴 게 하나...
3		연기 진짜 개 짙 구나 지루 할거 라고 생각 했는데...

[Step 2] 불용어(Stopwords) 제거 및 이중 리스트 토큰화

- 분석에 기여하지 않는 빈번한 조사, 접속사 등을 불용어로 정의하여 제거합니다.
- 각 문장을 형태소로 토큰화 함과 동시에 불용어를 필터링하여 이중 리스트 형태로 구축합니다.

Python Code

```
stopwords=['의','가','이','은','들','는','좀','잘','강','과','도','를','으로','자','에','와','한','하다']
temp = [word for word in tokenizer.morphs(sentence) if not word in stopwords]
tokenized.append(temp)
```

Rich Nested List Visualization

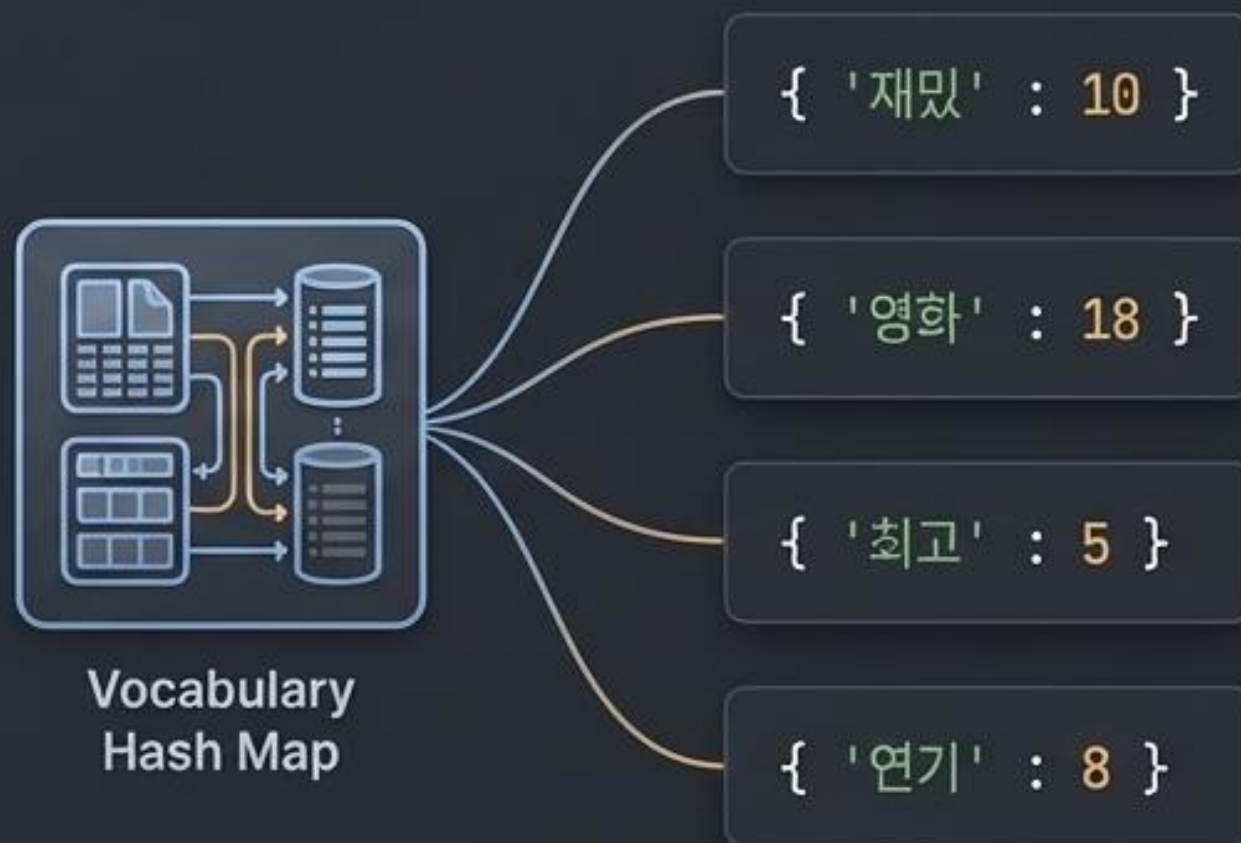
```
[
  ['어릴', '때', '보', '고', '지금', '다시', '봐도', '재밌', '어요', 'ㅋㅋ']
  ['디자인', '을', '배우', '학생', '외국', '디자이너', '그', '일군', '전통', '을', '통해', ...]
  ['폴리스', '스토리', '시리즈', '부터', '뉴', '까지', '버릴', '께', '하나', '없', '음', '최고']
  ... ['바보', '아니', '라', '병', '썬', '인', '듯']
]
```

[Step 2] 빈도수 분석: FreqDist를 활용한 단어 사전 구축

- NLTK의 FreqDist 모듈을 사용하여 1차원으로 병합된 토큰들의 등장 빈도수를 계산합니다.
- 단어 자체가 키(Key)가 되고, 등장 횟수가 값(Value)이 되는 해시맵(Hash Map) 구조를 생성합니다.
- 전체 샘플에서 고유 단어는 총 664개로 집계되었습니다.

Python Code

```
vocab = FreqDist(np.hstack(tokenized))  
print('단어 집합의 크기 : {}'.format(len(vocab))) # Output: 664  
  
vocab['재밋'] # Output: 10
```

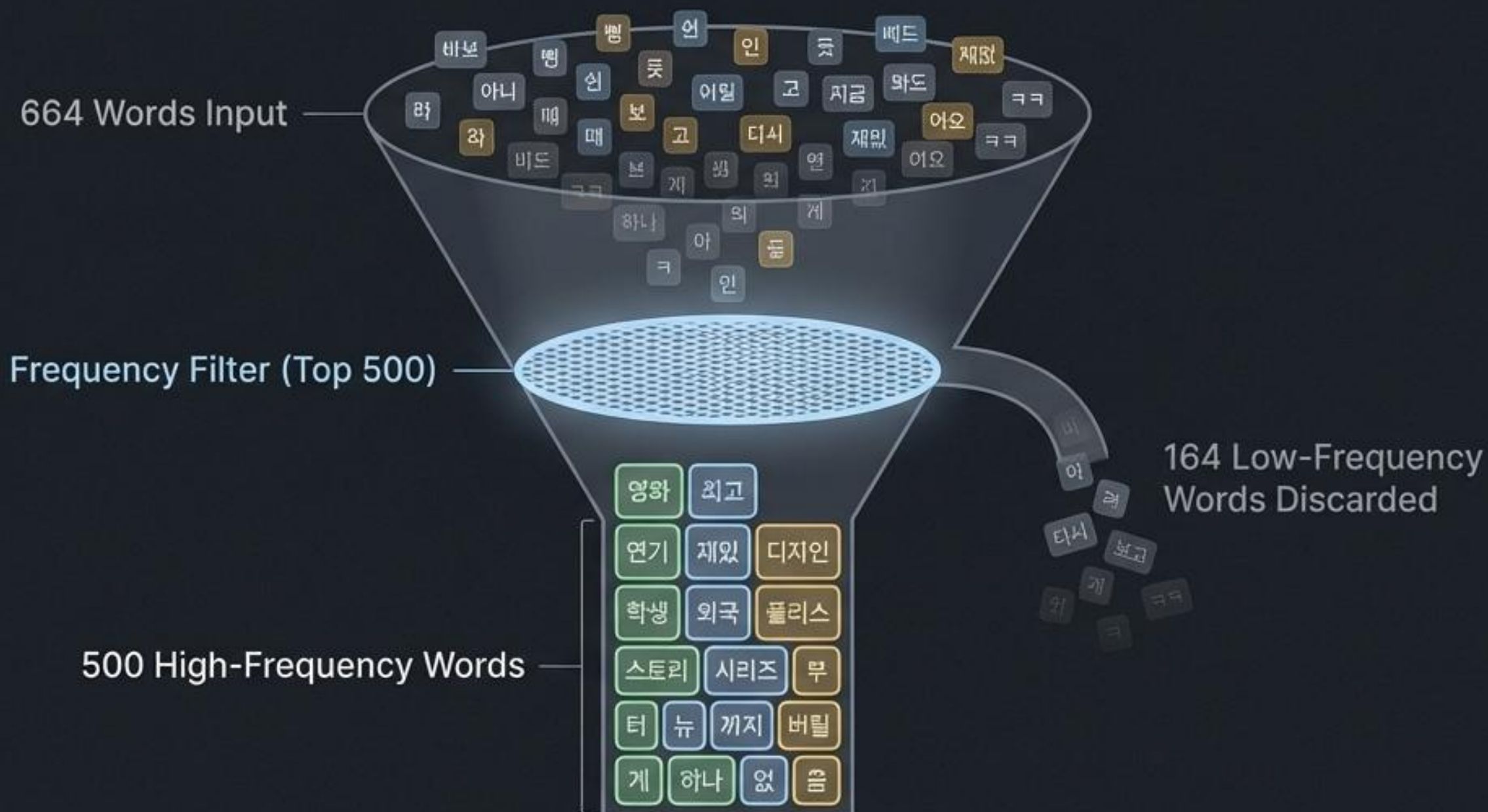


[Step 2] 빈도수 기반 단어 집합 크기 제한 (Truncation)

- 등장 빈도가 극히 낮은 희귀어나 오타는 기계 학습의 노이즈로 작용하여 연산량을 증가시킵니다.
- `most_common()` 함수를 활용해 빈도수 기준 상위 500개의 핵심 단어만 최종 집합으로 보존합니다.

Python Code

```
vocab_size = 500  
vocab = vocab.most_common(vocab_size)
```



[Step 3] 정수 인코딩 (Integer Encoding): 매핑 룰 정의

- 컴퓨터 연산을 위해 단어 집합의 각 텍스트를 고유한 정수(Integer) ID로 1:1 매핑합니다.
- 빈도수가 높은 순서대로 인덱스를 부여하되, 시스템에서 사용할 두 가지 특수 토큰을 위해 인덱스 0과 1을 사전 할당합니다.

Anatomy of an Encoding	
Index 0 →	[UNK] (Unknown: 단어 집합에 없는 단어)
Index 1 →	[PAD] (Padding: 길이 맞춤용 더미 토큰)
Index 2 →	[가장 빈도 높은 단어]
Index 3 →	[두 번째 빈도 높은 단어]
...	
Index 501 →	[500번째 단어]

Python Code	
<pre>word_to_index = {word[0] : index + 2 for index, word in enumerate(vocab)} word_to_index['pad'] = 1 word_to_index['unk'] = 0</pre>	

[Step 3] 정수 인코딩 실행 및 OOV (Out-Of-Vocabulary) 처리

- 원본 텍스트 리스트를 순회하며 사전을 참조하여 정수로 치환합니다.
- 크기 제한으로 사전에서 탈락한 단어(OOV)는 KeyError를 발생시키며, 이를 잡아 인덱스 0 (unk)으로 일괄 변환합니다.

Text Array Box

[['어릴', '때', '보', '고',
 '지금', '다시', '봐도',
 '재밌', '어요', 'ㅋㅋ'], ...]

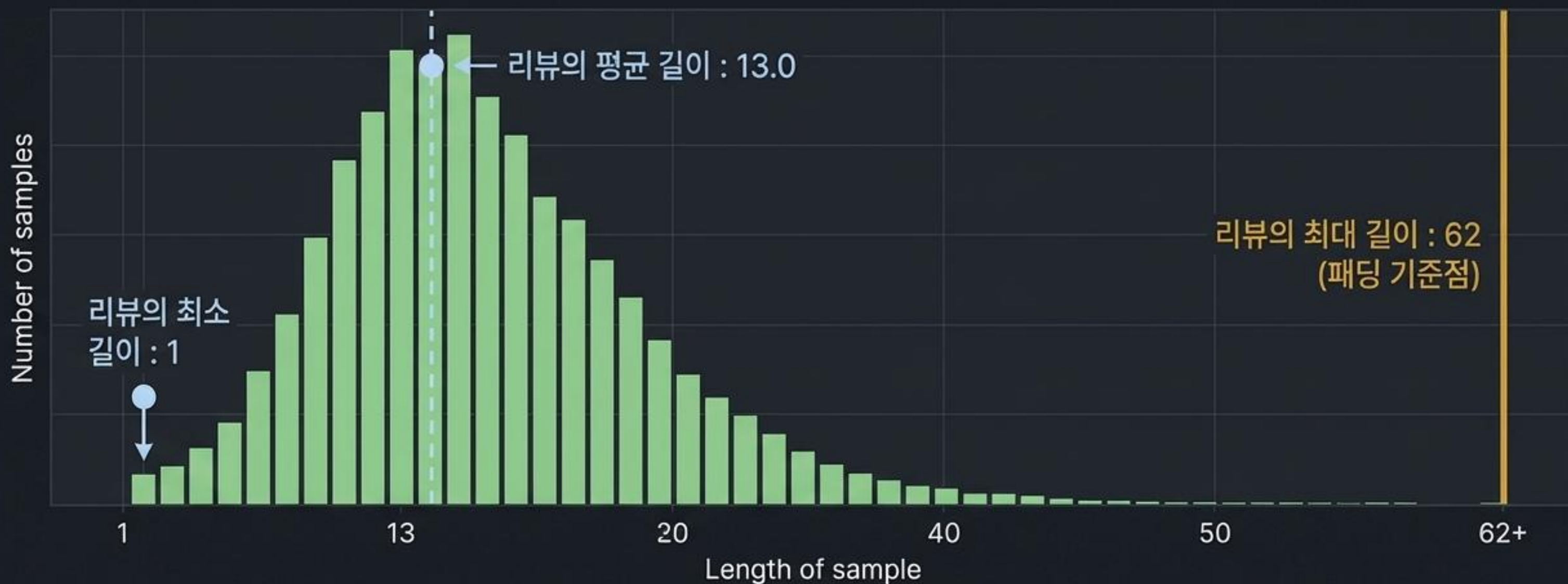
Integer Array Box

[[79, 27, 9, 4, 50, 42, 80,
 16, 28, 29],
 [188, 5, 81 ...]]

```
try:  
    temp.append(word_to_index[w])  
except KeyError:  
    temp.append(word_to_index['unk'])
```


[Step 4] 패딩(Padding)의 필요성: 시퀀스 길이 분포 분석

- 신경망 모델(텐서)은 동일한 규격의 입력 데이터를 요구합니다.
- 각 리뷰 문장의 길이는 제각각이므로, 기준이 될 최대 길이(max_len)를 산정하기 위해 분포를 파악합니다.



[Step 4] 패딩 실행: 가변 길이를 고정 길이로 규격화

- 샘플 리스트를 순회하며 지정된 최대 길이(max_len = 62)보다 짧은 샘플을 식별합니다.
- 부족한 길이만큼 [PAD] 토큰(인덱스 1)을 리스트 뒤에 반복해서 이어 붙입니다(Append).
- 처리 결과, 모든 리뷰의 길이가 62.0으로 완벽히 통일됩니다.



```
if len(line) < max_len:  
    line += [word_to_index['pad']] * (max_len - len(line))
```


[Step 4] 전처리 최종 산출물: 규격화된 정수 행렬(Matrix)

- 원본 텍스트가 신경망 모델에 주입되기 직전의 완전한 형태를 갖춘 직사각형 텐서(Tensor)입니다.
- 실제 정보(유의미한 정수)와 모델이 무시해야 할 빈 공간(1)이 구조적으로 명확히 분리되었습니다.

[	[	79	27	9	4	50	42	80	16	28	29	1	1	1	1	1	1	1...	],
	[	188	5	81	189	190	191	43	192	113	5	1	193	194	1	1	1	1...	],
	[	205	118	206	53	207	31	208	209	54	54	10	25	11	1	1	1	1...	]
	[	205	118	206	53	207	31	208	209	54	10	10	25	11	1	1	1	1...	]
	[	209	182	204	29	201	31	209	209	54	52	10	38	10	1	1	1	1...	]]

파이프라인 요약 및 다음 단계 (Vectorization)

- 파이프라인 요약: 비정형 텍스트 → 형태소 분리 → 사전 구축 → 정수 변환 → 길이 규격화
- 다음 단계 (Next Frontier): 현재의 정수 ID는 단순한 '이름표'일 뿐, 의미적 거리나 유사성을 담고 있지 않습니다.
- 이 정수들을 다차원 공간의 밀집 벡터로 변환하는 **워드 임베딩(Word Embedding)** 단계로 진입합니다.

